

at least when they first encounter them. This article is intended to help understand the power of generators and the way they promote expressing the 'separation of concerns'. As the Python documentation already covers generators very crisply, here we follow an example-driven approach to demonstrate the power of this concept.

Sample problem being solved

The problem used in the rest of the article is the 'Connect-4' game. This is a two-player game in which players take turns to drop chips into a vertical board, as shown in Figure 1.

Each player has a stack of chips of one colour. The player who first gets four 'connected' (adjacent) chips is the winner. The adjacency could be in a row, a column or a diagonal. A similar problem statement can be posed in other games, like the popular Android version of 'Jewels', where the idea is to find the longest successive connected sequence of beads of a colour.

The objective of the required software is to identify if a given board position represents a winning position. It needs to answer the following questions:

1. Does the board represent a winning position?
2. If yes, which colour is the winner and at which board location?

Solutions design

Niklaus Wirth, the designer of the Pascal programming language, made the following observation about programs:

`Programs = Algorithms + Data structures`

So, let us set out to work out both for the problem at hand.

Data structures

Here, the key data structure is the one that models the board, which we represent as a list of lists, where each sub-list represents the elements of a single row. Also, for the purpose of

this program, let us consider a 6 x 7 board; i.e., it has six rows and seven columns. With this board size, a sample Python list that represents the board is shown below:

```
board = [ [e00, e01, e02, e03, e04, e05, e06], # top most row
          [e10, e11, e12, e13, e14, e15, e16],
          ...
          [e50, e51, e52, e53, e54, e55, e56], # bottom most row
        ]
```

In our representation, board[i][j], represents the element at i+1-th row and j+1-th column, where rows are numbered from top to bottom and columns from left to right. I chose this ordering as it is very natural to the array ordering in programming languages, though we may sometimes count rows from bottom to top. Each element is either a blank (i.e., has no chip), or has a red or a yellow chip. For example, the bottom-most row shown in Figure 1 would be represented as:

```
[ 'r', 'y', 'y', 'y', 'r', 'y', 'r' ]
```

And the top-most row would be represented as all-blanks:

```
[ 'b', 'b', 'b', 'b', 'b', 'b', 'b' ]
```

The algorithm

The algorithm is straightforward – it is required to go over each row, column and diagonal to look for four connected chips of the same colour.

1. Walk each row to find a contiguous sequence of four chips of the same colour. If found, return the colour and location.
2. Walk each column to find a contiguous sequence of four chips of the same colour. If found, return the colour and location.
3. Walk each diagonal to find a contiguous sequence of four chips of the same colour. If found, return

the colour and location.

The first two of the above steps are self-explanatory but the third step merits some discussion as the diagonal referred to here does not mean just the two regular geometric diagonals of a rectangle. Instead, all the oblique sequences that are parallel to the geometric diagonals are to be considered (unless they have less than four elements) – so, it involves a set of diagonals. Also, diagonals from left to right are distinct from diagonals that go from right to left and hence, both sets have to be considered. With this understanding, the second version of our pseudo code is shown below:

1. Walk each row to find a contiguous sequence of four chips of the same colour. If found, return the colour and location.
2. Walk each column to find a contiguous sequence of four chips of the same colour. If found, return the colour and location.
3. Handling diagonals.
 - Walk each left-to-right diagonal contiguous sequence of four chips of the same colour. If found, return the colour and location.
 - Walk each right-to-left diagonal contiguous sequence of four chips of the same colour. If found, return the colour and location.

First attempt at the solution

This attempt does not use Python's generators but finds a reasonable solution. However, as we are interested in the process of developing code, the solution is presented in an iterative fashion. Initially, we develop the code just for handling rows and then extend it for checking within columns. Then, we refine the extended code to minimise redundancies.

The code for the first version that only handles rows in the board is shown below:

```
def compare_element(current, running, running_count):
```